

RETRIEVAL-AUGMENTED GENERATION (RAG) SYSTEM FOR ACADEMIC QUESTION ANSWERING

ABSTRACT

The rapid growth of digital academic resources has created challenges in efficiently retrieving and understanding relevant information. Traditional search systems often return documents rather than direct answers, requiring users to manually extract useful information. Large Language Models (LLMs) can generate human-like responses but may produce inaccurate or hallucinated information when operating without access to reliable sources.

This project presents a Retrieval-Augmented Generation (RAG) System for Academic Question Answering. The system combines document retrieval techniques with Large Language Models to provide accurate and context-aware answers from academic documents. Users can upload PDF documents, which are processed into text chunks and converted into vector embeddings. These embeddings are stored in a vector database for efficient retrieval. When a query is submitted, the system retrieves the most relevant document sections and supplies them to the language model for answer generation.

The proposed solution improves answer accuracy, reduces hallucinations, and provides source-based responses. A Streamlit-based web interface enables easy interaction between users and the system. The project demonstrates the practical application of modern Artificial Intelligence techniques in academic information retrieval and question answering.

Keywords: Retrieval-Augmented Generation, Large Language Models, Vector Database, FAISS, NLP, Academic Question Answering.

CHAPTER 1: INTRODUCTION

1.1 Background

Educational institutions increasingly rely on digital documents such as research papers, textbooks, lecture notes, and reports. Retrieving relevant information from large collections of documents is a difficult and time-consuming process.

Recent advances in Artificial Intelligence, particularly Large Language Models, have enabled machines to generate natural language responses. However, these models often rely solely on pre-trained knowledge and may generate inaccurate information.

Retrieval-Augmented Generation addresses this limitation by integrating information retrieval with language generation. Instead of relying only on model knowledge, the system retrieves relevant information from uploaded documents and uses it as context for answer generation.

1.2 Problem Statement

Students and researchers often spend significant time searching through lengthy academic documents to find specific information. Traditional keyword-based search systems may fail to understand semantic meaning and context.

The challenge is to build an intelligent system capable of understanding user queries, retrieving relevant academic content, and generating accurate answers with source references.

1.3 Objectives

The main objectives are:

1. Extract textual content from PDF documents.
 2. Generate vector embeddings from extracted text.
 3. Store embeddings in a searchable vector database.
 4. Retrieve relevant document chunks based on user queries.
 5. Generate context-aware answers using Large Language Models.
 6. Provide source references for generated answers.
 7. Develop a user-friendly web interface.
-

CHAPTER 2: LITERATURE REVIEW

Several studies have explored information retrieval and question-answering systems.

Traditional Information Retrieval systems rely on keyword matching techniques such as TF-IDF and BM25. While effective for simple searches, these approaches often fail to capture semantic relationships.

Transformer-based language models such as BERT, GPT, and LLaMA have significantly improved natural language understanding. However, standalone language models may generate inaccurate responses when queried about external documents.

Retrieval-Augmented Generation combines semantic retrieval and language generation, allowing systems to access relevant information during inference. Recent research shows that RAG architectures improve factual correctness and transparency.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Existing System

Existing search systems:

- Depend on keyword matching.
- Return entire documents rather than precise answers.
- Require manual reading and interpretation.
- Lack conversational interaction.

3.2 Proposed System

The proposed RAG system:

- Supports PDF document uploads.
- Uses semantic search instead of keyword matching.
- Retrieves contextually relevant information.
- Generates natural language answers.
- Provides source-based responses.

3.3 Advantages

- Higher answer accuracy.
 - Reduced hallucination.
 - Faster information retrieval.
 - User-friendly interaction.
 - Scalable architecture.
-

CHAPTER 4: SYSTEM DESIGN

4.1 Architecture Overview

The system consists of the following modules:

1. Document Upload Module
2. Text Extraction Module
3. Text Chunking Module
4. Embedding Generation Module
5. Vector Database Module
6. Retrieval Module
7. Language Model Module
8. User Interface Module

4.2 Workflow

Step 1: User uploads PDF documents.

Step 2: Text is extracted from PDFs.

Step 3: Text is divided into semantic chunks.

Step 4: Embeddings are generated.

Step 5: Embeddings are stored in FAISS.

Step 6: User enters a query.

Step 7: Query embedding is generated.

Step 8: Relevant chunks are retrieved.

Step 9: Retrieved context is passed to the LLM.

Step 10: Final answer is generated with references.

CHAPTER 5: TECHNOLOGIES USED

Programming Language

Python

Framework

Streamlit

Libraries

- LangChain
- FAISS
- PyPDF2
- Sentence Transformers
- Transformers
- NumPy
- Pandas

AI Models

- Embedding Models
 - Large Language Models
-

CHAPTER 6: IMPLEMENTATION

6.1 PDF Processing

PDF files are uploaded through the web interface. Text is extracted using PDF parsing libraries.

6.2 Text Chunking

Large documents are divided into smaller chunks to improve retrieval accuracy.

6.3 Embedding Generation

Each text chunk is converted into a numerical vector representation using sentence embedding models.

6.4 Vector Storage

Generated embeddings are stored inside FAISS for efficient similarity search.

6.5 Retrieval Process

User queries are converted into embeddings and compared against stored vectors.

6.6 Response Generation

Retrieved chunks are supplied to the language model as context. The model generates an answer using retrieved information.

CHAPTER 7: TESTING

Functional Testing

Test Case	Expected Result
Upload PDF	Successful Upload

Test Case	Expected Result
Text Extraction	Correct Text Output
Query Submission	Query Accepted
Retrieval	Relevant Chunks Returned
Answer Generation	Accurate Response

Performance Testing

- Retrieval Speed
- Response Accuracy
- User Satisfaction

CHAPTER 8: RESULTS AND DISCUSSION

The system successfully retrieves relevant academic information and generates context-aware answers.

Observed benefits include:

- Improved retrieval accuracy.
- Faster information access.
- Reduced misinformation.
- Better user experience.

The integration of retrieval and generation significantly enhances answer quality compared to standalone language models.

CHAPTER 9: FUTURE SCOPE

Future enhancements may include:

1. Multi-document comparison.
2. Support for images and tables.
3. Voice-based interaction.
4. Cloud deployment.
5. Multi-language support.
6. Advanced citation generation.
7. Research paper summarization.

CHAPTER 10: CONCLUSION

The Retrieval-Augmented Generation System for Academic Question Answering successfully combines semantic retrieval and language generation techniques. The system enables users to upload academic documents and obtain accurate, context-aware answers with source references.

By leveraging vector databases and Large Language Models, the proposed solution overcomes limitations of traditional search systems and standalone language models. The project demonstrates the effectiveness of Retrieval-Augmented Generation in academic information retrieval and provides a strong foundation for future intelligent educational applications.

REFERENCES

1. Lewis, P., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
2. Vaswani, A., et al. Attention Is All You Need.
3. LangChain Documentation.
4. FAISS Documentation.
5. Sentence Transformers Documentation.
6. Streamlit Documentation.
7. Research papers on Question Answering Systems and Information Retrieval.